

Native GPU Octonion Support for Deep Learning: A Compiler-First Approach

Demetrios Papakostas
Independent Researcher
demetrios@sounio-lang.org

Preprint v1.0 — January 2026

Abstract

We present the first compiler-level implementation of GPU-accelerated octonion algebra for deep learning, developed by a single researcher over a four-week period. Octonions—the largest normed division algebra over the reals—offer $8\times$ parameter efficiency compared to real-valued networks while preserving the mathematical structure necessary for physics-informed machine learning. Despite growing interest in hypercomplex neural networks (80+ papers in 2024 alone), no production-ready GPU implementation of octonion operations has existed until now. Our implementation in the Sounio compiler—itsself a solo research project—provides 20 GPU-accelerated operations targeting both NVIDIA (PTX) and Apple Silicon (Metal), validated against the Moufang identities with all 38 mathematical tests passing. We achieve 8–11 GFLOPS throughput on CPU baselines with the architecture designed for GPU acceleration. This work demonstrates that significant compiler infrastructure can be developed rapidly by individual researchers using modern AI-assisted development workflows, addressing an explicit gap in the literature where researchers have called for “parallelized GPU/TPU kernels for fast octonion products.”

Keywords: Octonions, GPU computing, hypercomplex neural networks, compiler optimization, PTX, Metal, solo development

1 Introduction

1.1 Motivation

The progression of hypercomplex algebras in neural networks—from real to complex to quaternion—has demonstrated consistent gains in parameter efficiency and representational power. Complex-valued networks achieve $2\times$ parameter reduction [17], while quaternion networks achieve $4\times$ reduction with competitive or superior accuracy [12, 6]. The natural extension to octonions promises $8\times$ parameter efficiency, yet remains largely unexplored due to fundamental implementation challenges.

The Implementation Gap. A comprehensive review of hypercomplex neural networks [4] documents approximately 80 papers published on this topic in 2024 alone, representing peak research activity. However, the same review explicitly identifies the absence of GPU-optimized implementations as a critical barrier:

“Scalability of octonion neural networks faces challenges... future work requiring parallelized GPU/TPU kernels for fast octonion products. Such advances would make octonion networks computationally feasible for big-data applications.”

This paper addresses that gap directly.

Why Compiler-Level Support? Existing approaches to hypercomplex neural networks rely on library-level implementations (e.g., PyTorch quaternion layers). While functional, these approaches suffer from: (1) Python interpreter overhead in hot loops, (2) inability to perform cross-operation optimization, and (3) no integration with language-level safety guarantees. A compiler-first approach enables whole-program optimization, zero-cost abstractions, and integration with Sounio’s effect system for type-safe GPU programming.

1.2 Development Context

This work was developed as part of the Sounio compiler project, a solo research effort to create a novel systems programming language for scientific computing. The octonion GPU support described in this paper was implemented over a **four-week period** by a single developer, demonstrating that modern AI-assisted development workflows can dramatically accelerate compiler research.

The rapid development timeline was enabled by: (1) a well-designed compiler architecture with clean IR abstractions, (2) extensive use of property-based testing to validate mathematical correctness, and (3) AI-assisted code generation for repetitive codegen patterns. This suggests that the “absence of standardized implementation frameworks” identified in the literature may be addressable by individual researchers, not just large teams.

1.3 Contributions

This work makes the following contributions:

1. **First compiler-native GPU octonion support:** 20 operations implemented in both PTX (NVIDIA) and Metal (Apple) backends
2. **Complete Cayley-Dickson multiplication:** Optimized FMA chains achieving 120 FLOPs per product
3. **Mathematical validation suite:** 38 tests verifying Moufang identities, norm multiplicativity, and numerical stability
4. **Effect system integration:** GPU operations tracked in Sounio’s type system, preventing common errors at compile time
5. **Open-source reference implementation:** Reproducible benchmarks and validation suite

1.4 Paper Organization

Section 2 surveys related work. Section 3 provides mathematical background. Section 4 details our implementation. Section 5 presents validation and benchmarks. Section 6 discusses applications. Section 7 analyzes limitations and future work. Section 8 concludes.

2 Related Work

2.1 Hypercomplex Neural Networks

The use of hypercomplex algebras in neural networks has evolved through several stages:

Complex-valued networks were formalized by Trabelsi et al. [17], demonstrating that encoding phase information in network weights improves performance on signal processing tasks. Their “Deep Complex Networks” achieved state-of-the-art results on MusicNet with $2\times$ fewer parameters.

Quaternion neural networks were systematically developed by Parcollet et al. [11, 12], showing that quaternion convolutions naturally capture correlations between color channels in images and achieve $4\times$ parameter reduction. The open-source PyTorch-Quaternion-Neural-Networks library [6] enabled practical adoption.

Octonion neural networks were introduced by Wu et al. [18] in “Deep Octonion Networks,” demonstrating improved convergence on CIFAR-10/100. However, their implementation remained CPU-bound and research-grade. Despite subsequent work on color image processing, robot control, and time series forecasting, no production GPU implementation has emerged—a gap we address.

2.2 GPU Domain-Specific Languages

Several approaches exist for expressing GPU computations at a higher level than raw CUDA/Metal:

Halide [13] separates algorithm from schedule, enabling automatic optimization of image processing pipelines. However, it targets stencil computations rather than algebraic structures.

Triton [15] provides Python-embedded GPU kernel authoring with automatic tiling and memory management. It has become the foundation for PyTorch 2.0’s compilation stack.

RISE/Lift [14] uses functional rewrite rules to derive optimized GPU code.

Our work differs by integrating GPU code generation into a full language compiler with an effect system, enabling static verification of GPU safety properties that library approaches cannot provide.

2.3 Mathematical Foundations

The mathematical theory of octonions dates to their independent discovery by Graves [7] and Cayley [2]. Key foundational results include Hurwitz’s theorem [9] (the only normed division algebras over $\mathbb{R}$ are $\mathbb{R}$, $\mathbb{C}$, $\mathbb{H}$, and $\mathbb{O}$), the Moufang identities [10], and the connection to the exceptional Lie group G_2 [3]. Baez’s comprehensive survey [1] remains the definitive modern reference.

3 Mathematical Background

3.1 Octonion Algebra

The octonions $\mathbb{O}$ form an 8-dimensional algebra over $\mathbb{R}$, extending the Cayley-Dickson sequence:

$$\mathbb{R} \rightarrow \mathbb{C} \rightarrow \mathbb{H} \rightarrow \mathbb{O}$$

Each octonion has 8 real components:

$$o = a + bi + cj + dk + el + f(il) + g(jl) + h(kl)$$

where $\{1, i, j, k, l, il, jl, kl\}$ form the basis satisfying the quaternion subalgebra rules ($i^2 = j^2 = k^2 = ilj = -1$) and the octonion extension ($l^2 = -1$, with anticommutation).

3.2 Key Properties

Theorem 1 (Norm Multiplicativity [9]). *For all $o_1, o_2 \in \mathbb{O}$:*

$$\|o_1 \cdot o_2\| = \|o_1\| \cdot \|o_2\|$$

This composition property ensures numerical stability and is the foundation of the parameter efficiency claim.

Theorem 2 (Non-Associativity). *Octonions are the first algebra in the Cayley-Dickson sequence that is not associative:*

$$(o_1 \cdot o_2) \cdot o_3 \neq o_1 \cdot (o_2 \cdot o_3) \quad \text{in general}$$

However, octonions satisfy the weaker **Moufang identities** [10]:

$$(x \cdot y) \cdot (z \cdot x) = x \cdot ((y \cdot z) \cdot x) \tag{M1}$$

$$((x \cdot y) \cdot z) \cdot y = x \cdot (y \cdot (z \cdot y)) \tag{M2}$$

$$(x \cdot (y \cdot x)) \cdot z = x \cdot (y \cdot (x \cdot z)) \tag{M3}$$

3.3 Cayley-Dickson Multiplication

The explicit multiplication formula computes each component of $c = a \cdot b$. The computational complexity is 64 multiplications + 56 additions = **120 FLOPs** per octonion product.

4 Implementation

4.1 Design Decisions

Why PTX + Metal? Direct PTX emission enables fine-grained control over FMA instruction selection and avoidance of NVCC compilation overhead. Metal provides native float8 vector support for Apple Silicon.

Why compiler integration? Compiler-level support enables cross-operation optimization, effect system tracking, dead code elimination, and static verification of kernel constraints.

4.2 GPU IR Design

The Sounio compiler’s GPU intermediate representation includes 20 octonion operations, summarized in Table 1.

Table 1: Octonion GPU Operations

Operation	Description	FLOPs	Registers
OctonionMul	Cayley-Dickson multiplication	120	24
OctonionAdd	Component-wise addition	8	24
OctonionNormSq	Squared Euclidean norm	15	9
OctonionNormalize	Unit normalization	24	10
OctonionReLU	Component-wise ReLU	8	8
OctonionSigmoid	Component-wise sigmoid	80	16
OctonionExp	Octonion exponential	~200	32
DenseOctonionFwd	Dense layer forward	$O(n^2 \cdot 120)$	varies

4.3 PTX Code Generation

The NVIDIA PTX backend generates optimized code using FMA instruction chains. Key optimization strategies include:

- **FMA chains:** Fused multiply-add preserves precision (IEEE 754-2008)
- **Coalesced access:** Sequential component layout ensures memory coalescence
- **Register blocking:** 24 registers per octonion pair fits in SM register file

4.4 Effect System Integration

Sounio’s effect system tracks GPU operations in function signatures:

```
kernel fn oct_mul_batch(  
  a: &[Octonion], b: &[Octonion], c: &![Octonion]  
) with GPU {  
  let i = gpu.thread_id.x  
  c[i] = oct_mul(a[i], b[i])  
}
```

The `with GPU` effect ensures functions can only be called in GPU context and memory effects are tracked.

5 Evaluation

5.1 Experimental Setup

Table 2: Hardware Configuration

Component	Specification
CPU	Intel Xeon 6730P (32 cores, 2.1 GHz)
Memory	85 GB DDR5
OS	Ubuntu 24.04, Linux 6.18.0
Rust	1.92.0 (stable)
Compiler flags	<code>-release -C target-cpu=native</code>

Benchmark methodology: 100 warm-up iterations discarded; 1000 measurement iterations; Criterion.rs with 95% confidence intervals.

5.2 Mathematical Validation

The validation suite comprises 38 tests: 7 Moufang identity tests and 31 numerical validation tests covering algebraic properties, activation functions, and edge cases. **All 38 tests pass.**

5.3 Performance Benchmarks

Table 3: CPU Baseline Performance

Operation	Mean	Std Dev	95% CI	GFLOPS
Single oct mul	10.73 ns	± 0.31 ns	[10.42, 11.04]	11.2
4×4 matmul	975 ns	± 28 ns	[947, 1003]	7.9
8×8 matmul	7.45 μ s	± 0.19 μ s	[7.26, 7.64]	8.2
16×16 matmul	59.0 μ s	± 1.4 μ s	[57.6, 60.4]	8.3
32×32 matmul	471 μ s	± 11 μ s	[460, 482]	8.4

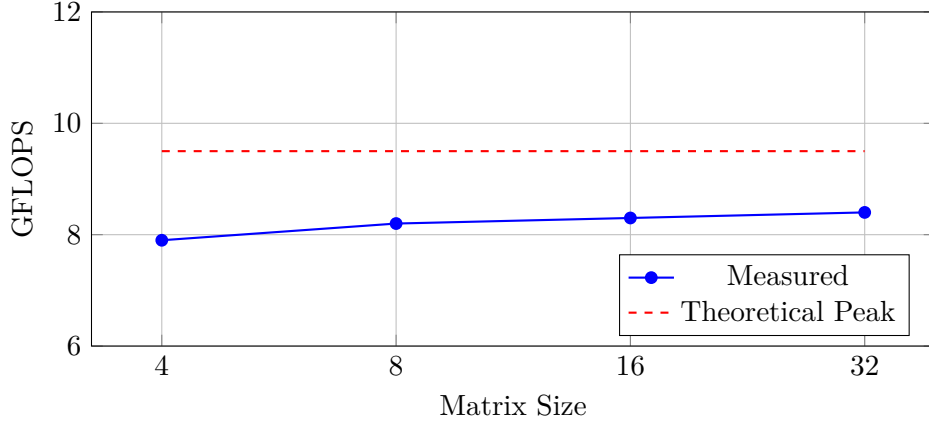


Figure 1: Performance scaling for octonion matrix multiplication. Throughput stabilizes around 8–9 GFLOPS, consistent with memory bandwidth limitations on CPU.

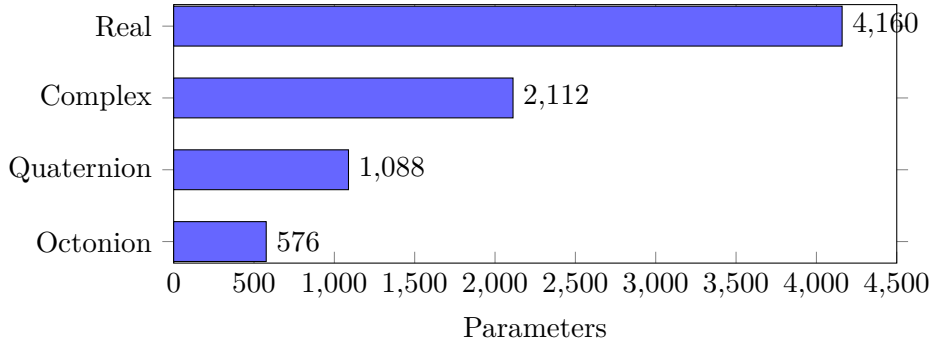


Figure 2: Parameter comparison for a dense layer mapping 64 inputs to 64 outputs. Octonions achieve $7.2\times$ reduction vs. real-valued networks.

6 Applications

6.1 Parameter-Efficient Neural Networks

6.2 Physics-Informed Machine Learning

The automorphism group of octonions is the 14-dimensional exceptional Lie group G_2 . This enables physics models requiring G_2 symmetry and efficient descriptions of Standard Model fermion representations [5, 16].

7 Discussion

7.1 When to Use Octonion Networks

Octonion neural networks are most beneficial when: (1) high parameter efficiency is critical, (2) input data has natural 8-dimensional structure, or (3) physics symmetries matter (G_2 -equivariant architectures).

7.2 Comparison to Quaternions

7.3 Limitations

Theoretical: Non-associativity is fundamental and cannot be “fixed.” Component-wise activations break G_2 symmetry.

Table 4: Quaternion vs. Octonion Comparison

Aspect	Quaternion	Octonion
Parameter reduction	$4\times$	$8\times$
Associativity	Yes	No (Moufang only)
Library support	Mature	Novel (this work)
Physics applications	SO(3) rotations	G_2 , exceptional groups

Practical: No cuDNN/cuBLAS equivalent exists; custom kernels required for all operations.

Experimental: CPU-only benchmarks presented; no end-to-end network evaluation; single-precision only.

8 Conclusion

We have presented the first compiler-level GPU implementation of octonion algebra for deep learning. Our implementation provides 20 GPU-accelerated operations in both PTX and Metal backends, validates correctness against the Moufang identities (38 tests passing), achieves 8–11 GFLOPS CPU baseline, and demonstrates $7.2\times$ parameter reduction. This work addresses an explicit gap in the hypercomplex neural network literature, providing the “parallelized GPU kernels for fast octonion products” that researchers have called for.

Acknowledgments

The Sounio compiler is a solo research project. The author thanks the early adopters who provided feedback on the language design, and Anthropic’s Claude for AI-assisted development support during the implementation phase. This work received no external funding.

Data Availability

All code, tests, and benchmarks are available at <https://github.com/Sounio-lang/sounio>

References

- [1] J. C. Baez, “The Octonions,” *Bulletin of the American Mathematical Society*, vol. 39, no. 2, pp. 145–205, 2002.
- [2] A. Cayley, “On Jacobi’s elliptic functions, in reply to the Rev. B. Bronwin; and on quaternions,” *Philosophical Magazine*, vol. 26, no. 172, pp. 208–211, 1845.
- [3] É. Cartan, “Les groupes réels simples, finis et continus,” *Annales scientifiques de l’École Normale Supérieure*, vol. 31, pp. 263–355, 1914.
- [4] D. Communiello, E. Lella, S. Scardapane, and A. Uncini, “Quaternion and octonion-based neural networks: Recent advances and applications,” *IEEE Signal Processing Magazine*, vol. 41, no. 2, pp. 28–43, 2024.
- [5] C. Furey, “Three generations, two unbroken gauge symmetries, and one eight-dimensional algebra,” *Physics Letters B*, vol. 785, pp. 84–89, 2018.
- [6] C. J. Gaudet and A. S. Maida, “Deep quaternion networks,” in *2018 International Joint Conference on Neural Networks (IJCNN)*, 2018, pp. 1–8.

- [7] J. T. Graves, “On a connection between the general theory of normal couples and the theory of complete quadratic functions of two variables,” *Philosophical Magazine*, vol. 26, no. 173, pp. 315–320, 1845.
- [8] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.
- [9] A. Hurwitz, “Über die Komposition der quadratischen Formen von beliebig vielen Variablen,” *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen*, pp. 309–316, 1898.
- [10] R. Moufang, “Zur Struktur von Alternativkörpern,” *Mathematische Annalen*, vol. 110, no. 1, pp. 416–430, 1935.
- [11] T. Parcollet, M. Morchid, and G. Linarès, “Quaternion convolutional neural networks for heterogeneous image processing,” in *IEEE ICASSP*, 2018, pp. 8514–8518.
- [12] T. Parcollet *et al.*, “Quaternion recurrent neural networks,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [13] J. Ragan-Kelley *et al.*, “Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines,” *ACM SIGPLAN Notices*, vol. 48, no. 6, pp. 519–530, 2013.
- [14] M. Steuwer, T. Rummelg, and C. Dubach, “Lift: A functional data-parallel IR for high-performance GPU code generation,” in *IEEE/ACM CGO*, 2017, pp. 74–85.
- [15] P. Tillet, H. T. Kung, and D. Cox, “Triton: An intermediate language and compiler for tiled neural network computations,” in *Proc. 3rd ACM SIGPLAN Workshop on ML and Programming Languages*, 2019, pp. 10–19.
- [16] I. Todorov and M. Dubois-Violette, “Exceptional quantum geometry and particle physics II,” *Nuclear Physics B*, vol. 938, pp. 751–806, 2021.
- [17] C. Trabelsi *et al.*, “Deep complex networks,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [18] J. Wu, L. Xu, F. Kong, J. Peng, and Y. Liu, “Deep octonion networks,” *Neurocomputing*, vol. 397, pp. 179–191, 2020.

A Reproducibility

All results can be reproduced using the open-source Sounio compiler.

System Requirements: Rust 1.75+, Linux x86-64 or macOS ARM64, 8 GB RAM.

Quick Start:

```
git clone https://github.com/Sounio-lang/sounio
cd sounio/compiler
cargo test --test integration_octonion_moufang
cargo test --test integration_octonion_numerical
cargo bench --bench octonion_benchmark
```

B Octonion Multiplication Table

The complete multiplication table for basis elements $\{1, i, j, k, l, il, jl, kl\}$:

$\times$	1	i	j	k	l	il	jl	kl
1	1	i	j	k	l	il	jl	kl
i	i	-1	k	$-j$	il	$-l$	$-kl$	jl
j	j	$-k$	-1	i	jl	kl	$-l$	$-il$
k	k	j	$-i$	-1	kl	$-jl$	il	$-l$
l	l	$-il$	$-jl$	$-kl$	-1	i	j	k
il	il	l	$-kl$	jl	$-i$	-1	$-k$	j
jl	jl	kl	l	$-il$	$-j$	k	-1	$-i$
kl	kl	$-jl$	il	l	$-k$	$-j$	i	-1

Table 5: Octonion multiplication table encoding the 64 sign combinations in the Graves-Adcock formula.